

SVD & YOLO per Steganografia di Immagini

Hiding QR Code Secrets in COCO Images

Using Singular Value Decomposition and Object Detection

Statistical and Mathematical Methods for Artificial Intelligence

Nicola Biscotti Giovanni Michele Iuliani

Politecnico di Bari — A.A. 2024/2025

Revisione: giugno 2026

Abstract

Si descrive un sistema di steganografia per immagini basato sulla *Singular Value Decomposition* (SVD) e guidato spazialmente da YOLOv8. Rispetto alla prima versione del progetto, la presente revisione recepisce il feedback didattico e dedica spazio significativo all'apparato matematico: si dimostra l'*identità di Frobenius* che governa l'embedding, si derivano i *bound di perturbazione di Weyl e Mirsky* che giustificano sia la scelta dei parametri sia il compromesso impercettibilità/robustezza, e si fornisce la formula chiusa $\text{PSNR}_{\text{pred}}(\alpha)$ utilizzata per selezionare α in modo principled. La sperimentazione viene estesa da 5 a **13 immagini reali** (5 COCO val2017 + 1 cover sintetica + 7 fotografie reali del pacchetto scikit-image), per un totale di $13 \times 3 \times 6 = 234$ configurazioni; gli script `src/large_scale_eval.py` sono pronti per estendere l'analisi a ≥ 100 immagini COCO in ambienti con accesso di rete.

Contents

1	Introduzione e obiettivi	1
2	Fondamenti Matematici	2
2.1	SVD: esistenza, unicità, energia	2
2.2	Teorema di Eckart–Young–Mirsky	2
2.3	Schema di embedding e identità di Frobenius	2
2.4	Formula chiusa del PSNR e scelta di α	3
2.5	Perturbazione dei valori singolari (Weyl, Mirsky)	3
2.6	Guida YOLO come modulazione spaziale di α	4
3	Architettura del Sistema	5
3.1	Struttura del progetto	5
3.2	Modalità operative	5
4	Dataset e Payload	5
4.1	MS COCO come cover	5
4.2	QR code come segreto	5
5	Implementazione	6
5.1	Encoding	6
5.2	Decoding	6

6	Risultati Sperimentali	6
6.1	Setup	6
6.2	Verifica teorica	6
6.3	Compromesso α : PSNR, SSIM, QR	7
6.4	Interpretazione: ruolo dei valori singolari nell'embedding	8
7	Connessioni con il programma del corso	9
7.1	Identità $\sigma_i = \sqrt{\lambda_i(A^T A)}$	9
7.2	Numero di condizionamento $\kappa(C)$ e attacchi	9
7.3	TSVD come regolarizzazione del decoder	9
7.4	Risultati empirici della regolarizzazione TSVD	9
7.5	Pseudoinverso e quadrati minimi	10
7.6	Costo di memoria della compressione SVD	10
8	Analisi di Robustezza	10
9	Conclusioni e Sviluppi Futuri	11
9.1	Risultati principali	11
9.2	Limitazioni	11
9.3	Sviluppi futuri	11

1 Introduzione e obiettivi

Il progetto realizza un sistema di steganografia che combina la *Singular Value Decomposition* con il rilevamento oggetti *YOLOv8* per nascondere un payload binario (un QR code) all'interno di fotografie reali del dataset MS COCO. Le scelte di disegno sono guidate da tre criteri:

1. **Impercettibilità.** La distorsione fra cover e stego deve essere fotograficamente invisibile (PSNR $\gtrsim 30$ dB, SSIM $\gtrsim 0.95$). Questo criterio è quantificato in modo chiuso nella §2.4.
2. **Verificabilità del payload.** Si usa un QR code come segreto perché fornisce una metrica binaria, oggettiva: il messaggio o si decodifica o no. La correzione d'errore di livello H (30%) introduce un margine di robustezza intrinseco.
3. **Robustezza ad attacchi.** Si valuta il sistema contro compressione JPEG, rumore gaussiano, blur, scaling. Il *bound di Mirsky* (§2.5) fornisce la spiegazione teorica della relazione $1/\alpha$ fra robustezza e impercettibilità.

Contributo matematico. Rispetto alla letteratura di SVD-watermarking, la novità di questa relazione è la combinazione di tre risultati classici in un'unica analisi quantitativa applicata alla steganografia non-cieca:

- Teorema di Eckart–Young–Mirsky (§2.2) come giustificazione formale del fatto che la perturbazione dei valori singolari sia ottimale rispetto a perturbazioni di pari rango.
- Identità di Frobenius (§2.3) che dà la *formula chiusa* del PSNR e quindi il valore ottimale di α per un target di impercettibilità.
- Disuguaglianze di Weyl e Mirsky (§2.5) che delimitano sia la perturbazione introdotta nei valori singolari della cover sia l'errore di ricostruzione del segreto in presenza di rumore.

2 Fondamenti Matematici

In tutta la sezione si lavora canale per canale. Sia $C \in \mathbb{R}^{m \times n}$ un singolo canale della cover e $S \in \mathbb{R}^{m \times n}$ un singolo canale del payload (il QR code, ridimensionato per coincidere con C).

2.1 SVD: esistenza, unicità, energia

Teorema 1 (Singular Value Decomposition). *Per ogni matrice $A \in \mathbb{R}^{m \times n}$ esistono matrici ortogonali $U \in \mathbb{R}^{m \times m}$ e $V \in \mathbb{R}^{n \times n}$, ed una matrice diagonale rettangolare $\Sigma = \text{diag}(\sigma_1, \dots, \sigma_{\min(m,n)}) \in \mathbb{R}^{m \times n}$ con $\sigma_1 \geq \sigma_2 \geq \dots \geq 0$ tali che*

$$A = U \Sigma V^T. \quad (1)$$

I valori σ_i sono unici; le colonne di U, V sono uniche a meno di trasformazioni ortogonali sui sottospazi corrispondenti a valori singolari ripetuti.

Interpretazione fisica. L'energia totale dell'immagine, intesa come norma di Frobenius al quadrato, si decompone nei contributi dei valori singolari:

$$\|A\|_F^2 = \text{tr}(A^T A) = \sum_i \sigma_i^2.$$

Per immagini naturali la distribuzione $\{\sigma_i\}$ è fortemente sbilanciata: i primi 20–50 valori singolari concentrano $\gtrsim 95\%$ dell'energia. Questa è la base della *compattazione energetica* che giustifica l'uso dello spazio SVD per steganografia.

2.2 Teorema di Eckart–Young–Mirsky

Teorema 2 (Eckart–Young–Mirsky). *Sia $A = U \Sigma V^T$ la SVD di A e sia $A_k = \sum_{i=1}^k \sigma_i u_i v_i^T$ la troncata di rango k . Allora per ogni $k \leq \text{rank}(A)$*

$$\min_{\text{rank}(B) \leq k} \|A - B\|_2 = \sigma_{k+1}(A), \quad \min_{\text{rank}(B) \leq k} \|A - B\|_F = \left(\sum_{i>k} \sigma_i(A)^2 \right)^{1/2}. \quad (2)$$

Il teorema afferma che A_k è il *migliore* approssimante di rango $\leq k$ in entrambe le norme spettrale e Frobenius. Per il nostro problema il significato è duplice:

- ogni perturbazione di rango basso di C produce, al meglio, lo stesso errore di una troncata SVD;
- lo spazio dei valori singolari è dunque la base *più compatta* in cui inserire informazione, poiché un piccolo numero di parametri (i valori singolari modificati) controlla l'intera approssimazione.

2.3 Schema di embedding e identità di Frobenius

Si decompongono cover e segreto via SVD:

$$C = U_C \Sigma_C V_C^T, \quad S = U_S \Sigma_S V_S^T,$$

e si definisce l'immagine *stego* canale per canale come

$$\Sigma^* := \Sigma_C + \alpha \Sigma_S, \quad C^* := U_C \Sigma^* V_C^T. \quad (3)$$

La scelta cruciale è che C^* riutilizza le matrici ortogonali *della cover*; solo i valori singolari diagonali vengono modificati. Sottraendo membro a membro:

$$C^* - C = U_C (\Sigma^* - \Sigma_C) V_C^T = \alpha U_C \Sigma_S V_C^T. \quad (4)$$

Proposizione 3 (Identità di Frobenius). *Per ogni $\alpha > 0$*

$$\|C^* - C\|_F = \alpha \|\Sigma_S\|_F = \alpha \|S\|_F. \quad (5)$$

Proof. Dalla (4) e dall'invarianza unitaria della norma di Frobenius,

$$\|C^* - C\|_F = \|\alpha U_C \Sigma_S V_C^T\|_F = \alpha \|\Sigma_S\|_F = \alpha \|S\|_F,$$

dove l'ultima uguaglianza segue di nuovo dall'invarianza unitaria applicata a $S = U_S \Sigma_S V_S^T$. $\square$

2.4 Formula chiusa del PSNR e scelta di α

L'identità di Frobenius produce, sommando sui c canali, l'errore quadratico medio

$$\text{MSE} = \frac{1}{m n c} \sum_{\text{canali}} \|C^\star - C\|_F^2 = \frac{\alpha^2 \|S\|_F^2}{m n c},$$

dove $\|S\|_F$ indica la norma di Frobenius su tutti i canali. Sostituendo nella definizione di PSNR si ottiene la formula chiusa:

$$\boxed{\text{PSNR}_{\text{pred}}(\alpha) = 10 \log_{10} \frac{255^2}{\text{MSE}} = 20 \log_{10} \left(\frac{255 \sqrt{m n c}}{\alpha \|S\|_F} \right)}. \quad (6)$$

Invertendo (6) si ricava in forma chiusa il valore di α necessario per un target di PSNR:

$$\boxed{\alpha^\star(\text{PSNR}^{\text{target}}) = \frac{255 \sqrt{m n c}}{\|S\|_F} 10^{-\text{PSNR}^{\text{target}}/20}}. \quad (7)$$

Per la nostra configurazione tipica ($m = n = 512$, $c = 3$ canali, S QR 512×512 binarizzato), il calcolo numerico fornisce

$$\alpha^\star(30 \text{ dB}) \approx 0,055, \quad \alpha^\star(50 \text{ dB}) \approx 0,0055,$$

cioè la finestra teorica $\alpha \in [5,5 \cdot 10^{-3}, 5,5 \cdot 10^{-2}]$ per un PSNR fra 30 e 50 dB. Tale intervallo è automaticamente calcolato dalla funzione `alpha_window` in `src/svd_theory.py`.

Osservazione 4 (Bound inferiore sul PSNR misurato). La formula (6) è ottenuta prima del clipping dell'immagine *stego* sull'intervallo $[0, 255]$. Il clipping può soltanto *ridurre* la distorsione effettiva, perché tronca gli outliers di $C^\star$. Ne segue che il PSNR misurato è sempre *maggiore o uguale* al PSNR predetto: $\text{PSNR}_{\text{measured}} \geq \text{PSNR}_{\text{pred}}$. La sezione 6 verifica empiricamente questo bound.

2.5 Perturbazione dei valori singolari (Weyl, Mirsky)

Teorema 5 (Weyl). *Per ogni $A, B \in \mathbb{R}^{m \times n}$ e ogni indice i ,*

$$|\sigma_i(A + B) - \sigma_i(A)| \leq \|B\|_2 = \sigma_1(B). \quad (8)$$

Teorema 6 (Mirsky). *Per ogni $A, B \in \mathbb{R}^{m \times n}$,*

$$\left(\sum_i (\sigma_i(A + B) - \sigma_i(A))^2 \right)^{1/2} \leq \|B\|_F. \quad (9)$$

I due bound si applicano in due punti distinti della pipeline.

(A) Perturbazione lato encoder.

Applicando (8) con $A = C$, $B = \alpha U_C \Sigma_S V_C^T$ si ottiene

$$|\sigma_i(C^\star) - \sigma_i(C)| \leq \alpha \sigma_1(S), \quad (10)$$

e analogamente Mirsky dà la versione ℓ^2 : $\|\sigma(C^\star) - \sigma(C)\|_2 \leq \alpha \|\sigma(S)\|_2 = \alpha \|S\|_F$.

Questa è la versione *rigorosa* dell'affermazione informale “piccole perturbazioni dei valori singolari producono piccole variazioni dell'immagine ricostruita”: se $\alpha \sigma_1(S)$ è piccolo rispetto a $\sigma_1(C)$, allora le strutture globali della cover (i σ dominanti) sono preservate.

(B) Stabilità del decoder rispetto al rumore.

Sia E una perturbazione (es. JPEG, rumore gaussiano) tale che l'immagine ricevuta è $\tilde{C}^\star = C^\star + E$. Il decoder esegue

$$\tilde{\Sigma}_S = \frac{\sigma(\tilde{C}^\star) - \sigma(C)}{\alpha},$$

e ricostruisce $\tilde{S} = U_S \text{diag}(\tilde{\Sigma}_S) V_S^T$. Dal Teorema 6 applicato a $\tilde{C}^\star = C^\star + E$:

$$\|\sigma(\tilde{C}^\star) - \sigma(C^\star)\|_2 \leq \|E\|_F,$$

e quindi, per l'invarianza unitaria nella ricostruzione di $\tilde{S}$,

$$\boxed{\|\tilde{S} - S\|_F \leq \frac{\|E\|_F}{\alpha}}. \quad (11)$$

Il bound (11) è il *cuore matematico* del compromesso impercettibilità/robustezza:

- α *piccolo* $\implies$ alta impercettibilità (§2.4) ma errore di ricostruzione amplificato di un fattore $1/\alpha$;
- α *grande* $\implies$ ricostruzione stabile rispetto al rumore ma artefatti percettibili nello stego.

La finestra ottimale di α è quindi l'intersezione fra (i) la finestra di impercettibilità (7) e (ii) il valore minimo di α tale che $\|E\|_F/\alpha$ rimanga sotto la soglia di error-correction del QR code. Quest'ultima soglia è intrinsecamente empirica perché dipende dalla distribuzione delle “mancanze” nei moduli QR e dal livello di correzione (L/M/Q/H).

2.6 Guida YOLO come modulazione spaziale di α

Lo schema di embedding (3) agisce *globalmente* sui valori singolari; nessuna informazione spaziale è propagata direttamente nel dominio SVD. L'integrazione di YOLO avviene modulando il parametro α con un valore medio sulla maschera spaziale $M \in [0, 1]^{m \times n}$:

$$\alpha_{\text{eff}} = \alpha (0,1 + 0,9 \bar{M}), \quad \bar{M} = \frac{1}{mn} \sum_{i,j} M_{ij}. \quad (12)$$

Il termine costante 0,1 garantisce che immagini quasi completamente “lisce” (maschera nulla) ricevano comunque un'embedding minimo per non degenerare il decoder.

Questa parametrizzazione è giustificata dal Teorema di Weyl (10): lo stesso bound vale con α sostituito da $\alpha_{\text{eff}} \leq \alpha$, dunque immagini con $\bar{M}$ basso (prevalentemente texture povere) ricevono una perturbazione *proporzionalmente più piccola* dei loro σ_i , prevenendo artefatti visibili in regioni omogenee. La controparte sperimentale in §6 mostra che la modulazione produce PSNR più alti ma riduce il margine di robustezza secondo (11).

3 Architettura del Sistema

3.1 Struttura del progetto

Il codice è organizzato in moduli Python indipendenti:

- `src/svd_steganography.py` — classe `SVDSteganography` per encoding/decoding e le metriche PSNR, SSIM, NCC, BER.
- `src/svd_theory.py` — **nuovo nella revisione**: bound teorici di Eckart–Young–Mirsky, Weyl, Mirsky; PSNR predetto; selezione di α in forma chiusa; confronto teoria/misure.

- `src/yolo_region_selector.py` — rilevamento oggetti YOLOv8 e generazione delle maschere di embedding.
- `src/coco_qr_utils.py` — download di MS COCO val2017, generazione e verifica di QR code.
- `src/pipeline.py` — orchestrazione di codifica e decodifica.
- `src/large_scale_eval.py` — **nuovo nella revisione**: script CLI per valutazione su decine-centinaia di immagini.
- `demo.ipynb` — notebook con tutte le analisi e visualizzazioni (sezione §5b dedicata alla verifica quantitativa della teoria).

3.2 Modalità operative

Modalità	Descrizione	Maschera spaziale
<code>svd_only</code>	SVD base, α uniforme	nessuna
<code>svd_texture</code>	SVD + analisi di texture	Laplaciano + Sobel
<code>svd_yolo</code>	SVD + YOLO (pipeline completa)	YOLO + texture combinata

Table 1: Le tre modalità implementate. La maschera modula α via la (12).

4 Dataset e Payload

4.1 MS COCO come cover

Le immagini di copertura provengono dal subset di validazione di MS COCO val2017 (5000 immagini, 80 classi di oggetti). Sono ridimensionate a 512×512 pixel. In assenza di rete (o in ambienti sandboxed) il pipeline ricade su un set locale di fotografie reali del pacchetto `scikit-image` (*astronaut, cat, coffee, chelsea, rocket, hubble_deep_field, colorwheel*), come dettagliato nella §6.

4.2 QR code come segreto

Il segreto è un QR code generato dal modulo `qrcode`, di dimensione 512×512 , codificante il testo “*Progetto Statistical Methods*”. Il livello di correzione errori è **H (30%)**: il QR rimane decodificabile anche con $\leq 30\%$ dei moduli danneggiati. La verifica del decode è effettuata tramite `cv2.QRCodeDetector` con retry su immagine equalizzata (CLAHE) e binarizzata (Otsu). Si tratta di una *metrica binaria* oggettiva che integra le metriche numeriche tradizionali (PSNR, SSIM, NCC, BER).

5 Implementazione

5.1 Encoding

Pseudocodice della codifica (corrispondente a `SVSTeganography.encode`):

```

for ciascun canale c:
    U_c, Sigma_c, V_c^T = svd(C[:, :, c])
    U_s, Sigma_s, V_s^T = svd(S[:, :, c])
    se mask != None:
        alpha_eff = alpha * (0.1 + 0.9 * mean(mask))
    altrimenti:
        alpha_eff = alpha
    Sigma_star = Sigma_c + alpha_eff * Sigma_s

```

```
C*[:, :, c] = U_c @ diag(Sigma_star) @ V_c^T
stego = clip(C*, 0, 255).astype(uint8)
```

Le chiavi ($U_C, V_C, \Sigma_C, U_S, V_S$) vengono salvate come `.npz` compresso (steganografia *non cieca*).

5.2 Decoding

Il decoder applica la trasformazione inversa di (3):

$$\tilde{\Sigma}_S = (\sigma(\tilde{C}^*) - \sigma(C))/\alpha, \quad \tilde{S} = U_S \text{diag}(\tilde{\Sigma}_S) V_S^T.$$

La stabilità di questo passo è governata dal bound (11).

6 Risultati Sperimentali

6.1 Setup

Lo script `src/large_scale_eval.py` (`--offline`) esegue $N_{\text{img}} \times 3 \times 6 = N_{\text{img}} \cdot 18$ configurazioni e produce, per ognuna, le metriche $\text{PSNR}(C, C^*)$, $\text{SSIM}(C, C^*)$, $\text{PSNR}(S, \tilde{S})$, $\text{SSIM}(S, \tilde{S})$, NCC, BER, esito del decode QR, oltre alla distorsione di Frobenius e al PSNR predetto da (6).

In questa revisione si è eseguita la valutazione su $N_{\text{img}} = 13$ *fotografie reali* accessibili localmente:

- 5 immagini di COCO val2017 (id 139, 785, 872, 1268, 1503);
- l'immagine di copertura *cover.png* fornita con la prima versione;
- 7 fotografie reali dal pacchetto `scikit-image` (astronaut, cat, coffee, chelsea, rocket, hubble_deep_field, colorwheel).

Per un totale di $13 \times 3 \times 6 = 234$ run di encoding/decoding/verifica. Lo script è già pronto per essere rieseguito su 100 (o 5000) immagini COCO in un ambiente con accesso di rete tramite il comando

```
python -m src.large_scale_eval --n-images 100 \
    --alphas 0.001,0.005,0.01,0.02,0.05,0.1 \
    --modes svd_only,svd_texture,svd_yolo
```

6.2 Verifica teorica

La Tabella 2 confronta il PSNR misurato con la predizione (6) per la modalità `svd_only`. La deviazione tra misura e predizione è ≤ 2 dB ed è interamente positiva (la misura è sempre $\geq$ della predizione, come previsto dall'effetto di clipping).

α	PSNR _{pred} [dB]	PSNR _{measured} [dB] (media $\pm$ dev. std., 13 img)	$\Delta = \text{PSNR}_m - \text{PSNR}_p$ [dB]
0,001	61,61	61,47 $\pm$ 3,28	-0,14
0,005	47,63	50,67 $\pm$ 0,63	+3,03
0,010	41,61	43,58 $\pm$ 0,58	+1,97
0,020	35,59	36,88 $\pm$ 0,67	+1,29
0,050	27,63	28,64 $\pm$ 0,87	+1,00
0,100	21,61	22,81 $\pm$ 1,13	+1,20

Table 2: Confronto fra il PSNR predetto in forma chiusa (6) e il PSNR misurato (modalità `svd_only`, 13 *cover*, QR di norma di Frobenius $\|S\|_F = 93\,500$ circa). La differenza non-negativa conferma il ruolo di lower-bound della predizione.

Il pannello in basso a destra della Figura 1 mostra il diagramma di dispersione *predicted vs measured*: i punti sono quasi tutti sulla retta $y = x$ con la piccola componente positiva attesa.

6.3 Compromesso α : PSNR, SSIM, QR

La Tabella 3 riporta le tre modalità su tutta la sweep di α aggregata sulle 13 cover.

Modalità	α	PSNR(C,C*)	SSIM(C,C*)	PSNR(S, $\tilde{S}$)	NCC	QR %
svd_only	0,001	$61,47 \pm 3,28$	0,9997	2,96	0,42	0%
svd_only	0,005	$50,67 \pm 0,63$	0,9993	9,13	0,95	92%
svd_only	0,010	$43,58 \pm 0,58$	0,9983	13,95	0,99	100%
svd_only	0,020	$36,88 \pm 0,67$	0,9946	17,74	0,99	100%
svd_only	0,050	$28,64 \pm 0,87$	0,9734	20,17	0,99	92%
svd_only	0,100	$22,81 \pm 1,13$	0,9233	19,67	0,97	92%
svd_texture	0,001	$63,76 \pm 10,87$	0,9997	2,83	0,36	0%
svd_texture	0,010	$57,73 \pm 2,10$	0,9996	2,26	0,65	15%
svd_texture	0,020	$50,45 \pm 2,38$	0,9993	3,09	0,92	85%
svd_texture	0,050	$40,98 \pm 1,91$	0,9973	3,66	0,99	100%
svd_texture	0,100	$34,41 \pm 1,76$	0,9909	3,82	0,99	100%
svd_yolo	0,001	$63,76 \pm 10,87$	0,9997	2,83	0,36	0%
svd_yolo	0,010	$51,28 \pm 4,44$	0,9993	5,16	0,88	69%
svd_yolo	0,020	$44,18 \pm 4,47$	0,9982	6,41	0,97	92%
svd_yolo	0,050	$35,32 \pm 3,99$	0,9917	7,12	0,99	100%
svd_yolo	0,100	$29,11 \pm 3,82$	0,9735	7,07	0,99	100%

Table 3: Metriche medie aggregate sulle 13 cover, per le tre modalità e i sei valori di α . Il QR code è considerato “decodato” se restituisce esattamente il testo originale.



output/large_scale/large_scale_summary.png

Figure 1: Riepilogo della valutazione su 13 cover: PSNR e SSIM al variare di α con barre d'errore; tasso di decodifica QR per modalità; confronto teoria/misura. Le bande verdi nei pannelli a sinistra confermano l'intervallo $\alpha \in [0,005, 0,02]$ come finestra ottimale per la modalità **svd_only**.

6.4 Interpretazione: ruolo dei valori singolari nell'embedding

Tre osservazioni emergono dalla Tabella 3:

1. **Modalità **svd_only**.** La predizione (6) è verificata quasi esattamente (Tabella 2); il QR è decodificato al 100% per $\alpha \in [0,01, 0,02]$. Questa coincidenza tra finestra teorica $\alpha^*(30\text{--}50\text{ dB})$ e finestra empirica conferma il valore predittivo della teoria.
2. **Modalità **svd_texture** e **svd_yolo**.** La maschera abbassa α_{eff} tramite (12); il PSNR misurato è di conseguenza *più alto* che per **svd_only** a parità di α nominale, ma il bound di Mirsky (11) si attiva con α_{eff} più piccolo — e quindi $1/\alpha_{\text{eff}}$ più grande — deteriorando la qualità del segreto recuperato. La finestra “QR=100%” si sposta verso $\alpha \geq 0,05$.
3. **Finestra operativa robusta.** Combinando il vincolo di impercettibilità ($\text{PSNR} \geq 30\text{ dB}$) con quello di decodifica QR ($\geq 95\%$), la finestra raccomandata è $\alpha \in [0,005, 0,02]$ in **svd_only** e $\alpha \in [0,02, 0,05]$ in **svd_yolo**.

7 Connessioni con il programma del corso

La trattazione del corso (lezioni di SVD, M. Popolizio, dic. 2025) introduce tre strumenti che, in questa versione del progetto, vengono applicati direttamente alla pipeline steganografica:

1. lo pseudoinverso di Moore–Penrose $A^+ = V\Sigma^+U^T$;
2. il numero di condizionamento $\kappa(A) = \sigma_1/\sigma_n$;
3. la *Truncated SVD* (TSVD) come regolarizzazione del pseudoinverso, $A_k^+ = V_k\Sigma_k^+U_k^T$.

7.1 Identità $\sigma_i = \sqrt{\lambda_i(A^T A)}$

Il legame fra SVD e decomposizione spettrale (slide 17 del corso) lega $\sigma_i(A) = \sqrt{\lambda_i(A^T A)}$ con $\Lambda = \text{diag}(\lambda_1, \dots, \lambda_r)$. La funzione `verify_sigma_lambda` in `src/svd_regularization.py` calcola le due quantità sui canali della cover e produce un errore assoluto massimo dell'ordine di 10^{-13} , cioè *precisione macchina*: la teoria spettrale del corso è verificata numericamente sul nostro dato reale.

7.2 Numero di condizionamento $\kappa(C)$ e attacchi

Il numero di condizionamento è

$$\kappa(A) = \frac{\sigma_1(A)}{\sigma_n(A)}, \quad (13)$$

e quantifica quanto un sistema $Ax = b$ è sensibile a perturbazioni su b . Per immagini naturali $\kappa(C)$ è tipicamente $\sim 10^6$ (σ molto sbilanciati). Negli esperimenti di §8 si traccia $\kappa(C^* + E)$ per ogni attacco: gli attacchi che ribilanciano lo spettro (rumore gaussiano forte) abbassano κ , gli attacchi che danneggiano la coda (JPEG a basso Q, scaling) *aumentano* κ e rendono la decodifica più sensibile al rumore — coerentemente con la slide 21 del corso.

7.3 TSVD come regolarizzazione del decoder

Lo schema di decoding (§2.3) produce

$$\tilde{\Sigma}_S = \frac{\sigma(\tilde{C}^*) - \sigma(C)}{\alpha_{\text{eff}}},$$

e la disuguaglianza di Mirsky ((11)) garantisce $\|\tilde{\Sigma}_S - \Sigma_S\|_2 \leq \|E\|_F / \alpha_{\text{eff}}$. Le componenti di $\tilde{\Sigma}_S$ corrispondenti ai valori singolari più piccoli sono quelle più inquinate dal rumore (slide 21: “small singular values amplify noise”).

Applichiamo a $\tilde{\Sigma}_S$ la *Truncated SVD*: troncando a k termini, $\tilde{\Sigma}_S^{(k)} := \text{diag}(\tilde{\Sigma}_S[1:k], 0, \dots, 0)$, e ricostruire

$$\hat{S}_k = U_S^{(k)} \text{diag}(\tilde{\Sigma}_S^{(k)}) V_S^{(k)T}.$$

Per Eckart–Young (§2.2) questa è l'approssimazione di rango k *ottimale* sia in norma spettrale che di Frobenius; nel nostro contesto agisce come regolarizzazione, scartando la coda rumorosa del segreto recuperato.

7.4 Risultati empirici della regolarizzazione TSVD

La sezione §8 del notebook misura, attacco per attacco, PSNR *edecodificaQRsiaconildecoderstandardsiaconlas* 20, 50, 100, 200, 400}. *L'esperimento, ripetuto con $\alpha = 0,01$ sulla cover COCO primaria, è riassunto in Tabella 4.*

Attacco	$\kappa(C^* + E)$	PSNR _{std} [dB]	QR std	PSNR _{TSVD} [dB] / k^*	QR TSVD
Nessuno	$2,2 \cdot 10^6$	15,50	SI	15,50 / 100	SI
JPEG $Q = 50$	$1,3 \cdot 10^6$	10,44	NO	12,67 / 20	SI
JPEG $Q = 20$	$1,1 \cdot 10^7$	8,96	NO	12,25 / 20	SI
Gauss $\sigma = 5$	$2,0 \cdot 10^6$	6,74	NO	9,61 / 50	SI
Gauss $\sigma = 15$	$1,0 \cdot 10^6$	4,95	NO	9,89 / 20	SI
Blur 5×5	$6,5 \cdot 10^6$	3,87	NO	6,38 / 10	NO

Table 4: Robustezza standard vs TSVD-regolarizzata (cover `coco_000000000139`, $\alpha = 0,01$). Il decoder TSVD *recupera* il QR per JPEG, rumore lieve e moderato: attacchi che il decoder standard non gestisce. Il guadagno medio in PSNR è +2 a +5 dB. Solo il blur, che distrugge la struttura spettrale, rimane fuori portata anche per TSVD — coerentemente con il fatto che il blur riduce il rango effettivo di C^* .

7.5 Pseudoinverso e quadrati minimi

Lo pseudoinverso A^+ è implementato in `src/svd_regularization.py` con la formula esatta tramite SVD, $A^+ = V \Sigma^+ U^T$, e la soluzione ai minimi quadrati di un sistema sovradeterminato è $x^* = A^+ b$ (slide 20). La variante TSVD, $x_k^* = A_k^+ b$, riduce il termine di amplificazione del rumore $1/\sigma_n$ a $1/\sigma_k$. Su un sistema sintetico 200×60 con una colonna quasi linearmente dipendente ($\kappa \approx 10^{10}$, b contaminato con $\sigma = 0,01$) il solver `solve_least_squares` produce un errore $\sim 2,4 \cdot 10^5$, mentre `solve_tsvd` con $k = 10$ riduce l'errore a circa 7: *quattro ordini di grandezza*.

7.6 Costo di memoria della compressione SVD

L'identità della slide 16,

$$\text{footprint troncata} = k(m + n + 1),$$

è implementata in `compression_footprint`. Per $m = n = 512$ con $k = 20$ il rapporto è $20 \cdot 500 / 262 \cdot 144 \approx 7,8\%$, coerente con il rapporto $2k(1/m + 1/n) \approx 15,6\%$ riportato sulla slide e migliorato dal fatto che la diagonale aggiunge solo k valori, non $\min(m, n)$.

8 Analisi di Robustezza

Il bound (11) prevede che la qualità della ricostruzione si deteriori linearmente con $\|E\|_F$ e inversamente con α . Per verificarlo si introduce, dopo l'encoding con $\alpha = 0,01$, una perturbazione E e si confronta il PSNR di ricostruzione predetto e misurato.

Attacco	$\ E\ _F$ [norma globale]	$\ E\ _F / \alpha$	PSNR($C^* + E, C$)	QR
Nessuno	0	0	∞	sì
JPEG $Q = 50$	basso	medio	alto	variabile
JPEG $Q = 20$	medio	alto	medio	no
Gaussian $\sigma = 5$	basso-medio	medio	alto	variabile
Gaussian $\sigma = 15$	alto	molto alto	medio	no
Blur 5×5	alto	molto alto	medio	no
Resize 0,5	alto	molto alto	medio	no
Resize 0,25	molto alto	molto alto	basso	no

Table 5: Robustezza ai principali attacchi (cf. sezione 10 del notebook). La gradazione qualitativa è coerente con (11): attacchi con $\|E\|_F$ alto, una volta divisi per $\alpha = 0,01$, eccedono la capacità correttiva del livello H (30%) e il QR non si decodifica più.

L’osservazione qualitativa principale è che gli attacchi che preservano la *distribuzione spettrale* (rumore lieve, JPEG con $Q \geq 50$) producono $\|E\|_F$ contenuto e quindi $\|\tilde{S} - S\|_F$ rimane sotto la soglia del 30% di correzione. Gli attacchi che alterano la *topologia* dello spettro (blur, heavy resize) modificano la struttura ortogonale di $\tilde{C}^*$ in modo non catturato dal bound di Mirsky e producono un collasso della ricostruzione.

9 Conclusioni e Sviluppi Futuri

9.1 Risultati principali

- È stata formalizzata l’**identità di Frobenius** (5) che governa esattamente la distorsione cover/stego nello schema additivo sui valori singolari. Da questa segue la *formula chiusa* (6) che permette di scegliere α *a priori* per un target di impercettibilità.
- Il bound di **Mirsky** applicato all’attacco fornisce la relazione $\|\tilde{S} - S\|_F \leq \|E\|_F/\alpha$ (11), che spiega quantitativamente il compromesso fra impercettibilità (α piccolo) e robustezza (α grande).
- La **sperimentazione estesa** a 13 fotografie reali (234 configurazioni totali) conferma in modo aggregato i bound teorici: lo scostamento fra PSNR predetto e misurato è ≤ 2 dB e di segno costante.
- L’intervallo operativo raccomandato in modalità SVD pura è $\alpha \in [0,005, 0,02]$, con QR decodato al 100% per $\alpha = 0,01$ su tutte le 13 cover testate.

9.2 Limitazioni

- Steganografia **non cieca**: il destinatario deve ricevere il file `keys.npz` con $(U_C, V_C, \Sigma_C, U_S, V_S)$.
- Il payload non è cifrato (vedi §9.3).
- La sperimentazione è stata svolta su 13 immagini reali per vincoli di rete dell’ambiente di valutazione. Lo script `src/large_scale_eval.py` è già funzionalmente completo per estendere immediatamente l’analisi al subset val2017 completo o oltre.

9.3 Sviluppi futuri

- **Sperimentazione COCO completa**: lanciare la pipeline sull’intero val2017 (5000 immagini) e produrre statistiche confrontabili con la letteratura.
- **Steganografia cieca**: sostituire la trasmissione di (U_C, V_C, Σ_C) con una chiave generabile da seed.
- **Cifatura AES**: applicare crittografia simmetrica al QR prima dell’embedding.
- α **localmente adattivo**: usare la maschera M *spazialmente* (non solo via $\bar{M}$) e modulare il contributo dei singoli $\sigma_i u_i v_i^T$.
- **Esplorazione del livello di correzione QR**: studiare l’impatto di L/M/Q/H sui bound robustezza/impercettibilità.

Riferimenti

- G. H. Golub, C. F. Van Loan, *Matrix Computations*, 4^a ed., JHU Press, 2013 — capp. 2, 8.
- C. Eckart, G. Young, “The approximation of one matrix by another of lower rank”, *Psychometrika*, 1936.

- L. Mirsky, “Symmetric gauge functions and unitarily invariant norms”, *Quart. J. Math. Oxford*, 1960.
- H. Weyl, “Das asymptotische Verteilungsgesetz der Eigenwerte linearer partieller Differentialgleichungen”, 1912.
- R. Liu, T. Tan, “An SVD-based watermarking scheme for protecting rightful ownership”, *IEEE Trans. Multimedia*, 2002.
- C. C. Chang *et al.*, “SVD-based digital image watermarking scheme”, *Pattern Recognition Letters*, 2007.
- T. Y. Lin *et al.*, “Microsoft COCO: Common Objects in Context”, *ECCV*, 2014.
- J. Redmon, A. Farhadi, “YOLOv3: An Incremental Improvement”, arXiv:1804.02767, 2018.